Spectral Methods for Poiseuille Flow in 2D

Jai Singh Sagoo

I. Introduction

Spectral methods have been extensively used to analyze fluid dynamics problems, from stability analysis to simulation of different scenarios. This paper will focus on using a Fourier spectral method to solve Navier-Stokes' equations in a 2D channel (Poiseuille flow) which will then be used to model blood flow through a small channel, representing a capillary.

The simulation will focus on a general Poiseuille flow simulation with variable pipe radius, allowing a variety of scenarios to be modelled, such as large-scale channel flow and variable flow rates. The implementation section of the paper will then simulate flow through a small channel ($R \approx 10^{-5}m$) and a slightly larger channel ($R \approx 10^{-3}m$), with no-slip boundary conditions and a variable pressure function to approximate a heartbeat. In Sang-Suk Lee et al. [1] the blood flow velocity in the fingers and the radial artery are measured, which I used as the primary data set to test the model.

If the simulation is accurate, the observed blood flow should be within a certain bound around the observed flow rates in the experiment. The simulation by Valerie Phi Van et al. uses the same model for calculating the flow rate in blood vessels of mouse brains, albeit without a variable pressure function. This showed reasonably accurate results, with the simulated blood flow in the upper range of the measured flow rate, which is likely due to assumptions such as a Newtonian fluid and steady flow. Despite this, the fact that the simulation gave velocities in the range measured indicates a sufficiently accurate model to motivate this simulation.

II. Theory

The equation for pseudo-Poiseuille flow (with time-dependency) can be derived from the Navier-Stokes' equation:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f} \tag{1}$$

Where $\mathbf{u} = \mathbf{u}(x, y, t)$ and $p = p(x, y, t)$.

- We have no body forces, so $\mathbf{f} = 0$
- Incompressibility means $\nabla \cdot \mathbf{u} = 0$

In normal Poiseuille flow, steady flow means that $\frac{\partial \mathbf{u}}{\partial t} = 0$, but since we are dealing with $p = p(t)$, flow is no longer steady and the term remains.

This leaves us with:

$$\frac{\partial \mathbf{u}}{\partial t} = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{u} \tag{2}$$

(as opposed to $\frac{1}{\rho}\nabla p = \nu \nabla^2 \mathbf{u}$). For 2D pseudo-Poiseuille flow, we have the constraints $x \in R,\ y \in [-A, A],\ t \geq 0$

A. Pressure Function Construction

To construct the pressure function, we need to define the bounds and expected behaviour of the pressure function, such as:

Periodicity:

$$p_{x_0,y_0}(t) = p_{x_0,y_0}(t + \phi)$$

where $\phi \in R$.

Non-negativity:

$$p(x, y, t) \geq 0 \quad \forall\ x \in R,\ y \in [-A, A],\ \&\ t \geq 0$$

Spatial dependency:

The setup of the problem necessitates that $\frac{\partial p}{\partial y} = 0$, as there are no forces acting vertically. This leaves us with $p(x, y, t) = p(x, t)$.

Now, if we take (2) and apply the divergence operator:

$$\nabla \cdot \frac{\partial u}{\partial t} = \nabla \cdot \left(-\frac{1}{\rho} \nabla p + \nu \nabla^2 u \right)$$

$$= -\frac{1}{\rho} \nabla^2 p + \nu \nabla \cdot \left(\nabla^2 u \right)$$

$$= -\frac{1}{\rho} \nabla^2 p + \nu \nabla^2 \left(\nabla \cdot u \right) = \frac{\partial}{\partial t} \left(\nabla \cdot u \right)$$

Using the incompressibility condition, $\nabla \cdot u = 0$, this leaves us with:

$$-\frac{1}{\rho} \nabla^2 p = 0 = \nabla^2 p \tag{3}$$

(3) gives us the final condition for P that we can use to construct a pressure function. This means that p is a harmonic scalar field, as it solves the Laplace equation (3). Since $p(y)$ is defined on $[-A, A]$, Liouville's theorem doesn't apply and we can have a non-constant function but as stated above this is not necessary as p has no y dependency.

Hence, for this paper, we define:

$$\nabla p(x,t) = G f(t)$$

where $G \in R$ and $f(t) \geq 0$ is a sinusoidal function approximating the pressure applied by the heart as it beats.

$$\Rightarrow p(x,t) = G x f(t)$$

Which satisfies the condition that $\nabla^2 p = 0$ as Gx is a harmonic function.

B. Stability Analysis

If we add perturbations to $\mathbf{u}$ and p,

$$\mathbf{u} = \mathbf{u} + \mathbf{u}', \quad p = p + p'$$

and substitute into equation (2), we get the following perturbation equation:

$$\frac{\partial (\mathbf{U} + \mathbf{u}')}{\partial t} = -\frac{1}{\rho} \nabla (p + p') + \nu \nabla^2 (\mathbf{U} + \mathbf{u}')$$

Linearizing this perturbation equation results in the following linearized pseudo-Poiseuille flow equation:

$$\frac{\partial \mathbf{u}'}{\partial t} = -\frac{1}{\rho}\nabla p' + \nu\nabla^2\mathbf{u}'$$

Defining the perturbation $p' = Gx\sin^2(\omega t)$:

$$\frac{\partial \mathbf{u}'}{\partial t} = -\frac{1}{\rho}G\sin^2(\omega t) + \nu\nabla^2\mathbf{u}' \tag{4}$$

Since $\mathbf{u}'$ is differentiable, it is sufficiently smooth and therefore continuous, meaning that we can take the Laplace transform of (4) in time. Because Laplace transforms are linear transformations, we can apply them to each term in the equation successively:

$$\mathcal{L}[\frac{\partial \mathbf{u}}{\partial t}] = \mathcal{L}[-\frac{1}{\rho}G\sin^2(\omega t)] + \mathcal{L}[\nu\nabla^2\mathbf{u}]$$

$$\Rightarrow sU'(x,y,s) - u_0'(x,y) = -\frac{G}{\rho}\frac{\omega^2}{2(s^2+4\omega^2)} + \nu\nabla^2U'(x,y,s)$$

where $u_0'(x,y) = u'(x,y,0)$.

$$\Rightarrow (s-\nu\nabla^2)U'(x,y,s) = -\frac{G}{\rho}\frac{\omega^2}{2(s^2+4\omega^2)} + u_0'(x,y)$$

If we assume $U' = \hat{U}(y,s)e^{i\alpha x}$ then we can use the normal mode to analyze the eigenvalues of the PDE. Substituting in we get:

$$\Rightarrow \left(s + \alpha^2\nu - \nu\frac{\partial}{\partial y}\right)e^{i\alpha x}\hat{U} = -\frac{G}{\rho}\frac{\omega^2}{2(s^2+4\omega^2)} + \hat{U}(y,0)e^{i\alpha x} \tag{5}$$

Next, we define the eigen-decomposition of $\hat{U} = \sum c_n\phi_n$, where $c_n = c_n(s)$ are the amplitudes of the n^{th} mode and $\phi_n = \phi_n(y)$ are eigenfunctions satisfying:

$$(\alpha^2\nu - \nu\frac{\partial}{\partial y})\phi_n = \lambda_n\phi_n$$

Hence we can rewrite (5) as:

$$(s+\nu\lambda_n)\,c_n(s)\phi_n = -\frac{G}{\rho}\frac{\omega^2}{2(s^2+4\omega^2)} + c_n(0)\phi_n$$

To see if the system is stable or not, we can rearrange for $\nu\lambda_n$:

$$\nu\lambda_n = -s + \frac{-\frac{G}{\rho}\frac{\omega^2}{2(s^2+4\omega^2)} + c_n(0)\phi_n(y)}{c_n(s)\phi_n(y)} = -s + \frac{c_n(0)}{c_n(s)} - \frac{1}{c_n(s)\phi_n(y)}\frac{G}{\rho}\frac{\omega^2}{2(s^2+4\omega^2)}$$

where to avoid division by 0, we must consider $y \in (-A, A)$ as $\phi_n(A) = \phi_n(-A) = 0$. (We could also consider partial slip to avoid zero division)

For the system to be stable, we must have $\mathbf{Re}(\lambda_n) < 0$, $\forall n \in N$. As $s \to \infty$, the first term, $-s$, will dominate the other two, enforcing $\nu\lambda_n < 0$ and therefore the stability of the system. This also gives us the effect of larger dynamic viscosity, ν, increasing the rate at which the perturbed system stabilizes.

These effects also align with our physical intuition of how a fluid works, as we should expect that a more viscous system experiences significantly less turbulence and exhibits more stability when perturbed from steady flow. For blood flow, we know that it is non-Newtonian but if we take the typical viscosity in a capillary of diameter $D = 2A$, we can use Poiseuille's law to calculate the apparent viscosity of blood as done in T.W.Secomb, 2009 [2]:

$$\nu_{app} = \frac{\pi}{128}\frac{\Delta p D^4}{LQ} \tag{6}$$

Where Δp is the pressure change over some distance, L, and Q is the volumetric flow rate. Since Δp and Q are time-dependent, we can approximate by taking the average of both as $p_{avg} = \frac{1}{2}p_{max}$ and $Q_{avg} = \frac{1}{2}Q_{max}$ because the pressure functions we are considering are sinusoidal with only a single mode.

III. Implementation

A. Spectral Method Implementation

When implementing the numerical solver, we must find an accurate way to solve the system. If we were dealing with an elliptic PDE, such as for traditional Poiseuille flow, it would be possible to use finite difference methods. However, since we are dealing with a parabolic partial-differential equation (with time dependency), we are forced to use another method.

Spectral methods are best suited for this application as they allow us to convert the PDE, (2) into an ODE in time by taking the Fourier transform of both sides and then

using a method such as Numpy's Runge-Kutta based RK45 solver [3]. This will ensure greater accuracy in the spatial domain and simplify the calculation.

We can do this analytically to see how the spatial dependence is removed from the equation. Starting with (2) and applying the Fourier transform gives:

$$\mathcal{F}[\frac{\partial \mathbf{u}}{\partial t}] = \mathcal{F}[-\frac{1}{\rho}\nabla p] + \mathcal{F}[\nu\nabla^2\mathbf{u}]$$

$$= \mathcal{F}[-\frac{1}{\rho}\nabla p] + \mathcal{F}[\nu\nabla^2\hat{\mathbf{u}}] = \mathcal{F}[-\frac{1}{\rho}G\sin(\omega t)] + \mathcal{F}[\nu\nabla^2\hat{\mathbf{u}}]$$

$$= \frac{d\hat{\mathbf{u}}}{dt} = -\frac{1}{\rho}G\sin(\omega t)\delta(k) - \nu k^2\hat{\mathbf{u}} \tag{7}$$

Now that we have transformed into Fourier space, we can solve the ODE, (7), analytically using the SciPy 5^{th} order RK45 solver. This method provides a number of advantages in computational complexity as well as accuracy:

- Convergence: Spectral methods offer exponential convergence rates for smooth solutions instead of polynomial convergence rates seen in traditional finite difference schemes. Especially for Navier-Stokes' equations, which have smooth solutions for non-turbulent flows.

- Complexity: When using the FFT, we reduce the computational complexity of the system from $\mathcal{O}(N^2)$ to $\mathcal{O}(N\log(N))$, allowing for faster/more efficient calculation. This allows us to have higher resolution simulations, thereby capturing more detail in the solution, such as possible boundary effects

- Boundary conditions: Fourier transforms naturally incorporate the boundary conditions $\mathbf{u}(A) = \mathbf{u}(-A)$ as they represent functions as a sum of sin and cos functions, which are themselves periodic.

- Incompressibility: When we take the Fourier transform of a system, we project it onto a domain where $\mathcal{F}[\nabla\cdot\mathbf{v}] = \mathbf{k}\cdot\hat{\mathbf{v}} = 0$, so the dot product of a function $\hat{\mathbf{v}}$ with $\mathbf{k}$ must be 0, so $\mathbf{v}$ is essentially projected onto a plane orthogonal to $\mathbf{k}$.

However, there are some disadvantages to using spectral methods, such as if we are not on a periodic domain then we are forced to use another such as the Chebyshev spectral method, which involves decomposing the system using Chebyshev polynomials, but this negates the computational advantage of the FFT having complexity of $\mathcal{O}(n\log n)$.

B. Code

I defined constants such as ν and G outside the functions for computing the FFT, IFFT and solving the transformed ODE to make running the simulation with different parameters easier. I also chose to use SI units to make sure the simulation accurately reflects what we expect to observe on small timescales.

I defined the pressure function as before in (4):

```
def pressure(w, t):
    return np.sin(w*t)**2
```

This way, instead of defining a discrete array for $\sin(\omega t)$, which can change for different ω, number of time-steps and final time, T_{final}, the pressure is calculated as a function. The next function sets up the Navier-Stokes' equation in Fourier space so that we can solve the IVP using the SciPy RK45 Runge-Kutta scheme:

```
#defines the RHS of the ODE in Fourier space
def navierstokes_fourier(t, u_hat):
    #compute the second derivative (laplacian) in fourier space
    u_y2_hat = -(k**2) * u_hat

    #define du_hat/dt = G/rho * sin^2(wt) + nu*(-k^2 u_hat)
    du_hat_dt = (G * pressure(omega, t)) / rho + nu * u_y2_hat
    return du_hat_dt

#uses RK45 to solve the 1st order time ODE
solution = solve_ivp(navierstokes_fourier, [0, T], u0_hat, method = '
   RK45', t_eval = np.arange(0, T, dt))
```

Now that we have calculated $\hat{\mathbf{u}}$, we can use the inverse Fourier transform to solve for $\mathbf{u}$ in real space:

```
#inverse fourier transform to get the velocity in real space
u_solution = np.array([sci.fft.ifft(u_hat).real for u_hat in solution.y.
   T])
```

C. Preliminary Testing

To check for accuracy, we can compare the generated solutions to what we intuitively predict for this system (Fig. 1).

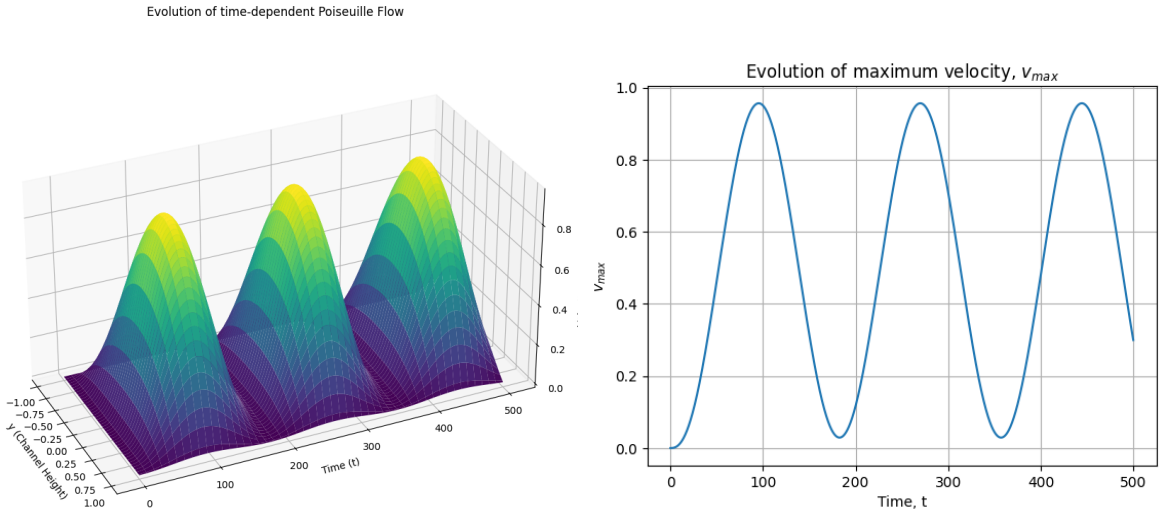

Figure 1: Evolution of velocity profiles over time

Due to the oscillating pressure term, $\nabla p = G\sin^2(\omega t)$, we can see the velocity and volume flux oscillating, with the boundary conditions of $\mathbf{u}(A) = \mathbf{u}(-A) = 0$ still enforced, causing the velocity profile at some time $t = t_{current}$ to look like the standard quadratic solution for Poiseuille flow (Fig.1 left). When analyzing the evolution of maximum velocity, over time, we also see that after $t = 0$, $v_{max} > 0$ (Fig.1 right).

D. Results

Sang Suk Lee et. al [1] found that for a people with a 'healthy' BMI (between $18.7 \sim 23.5\,kgm^{-3}$), the blood velocity (BV) in finger capillaries was between $6.3 \sim 8.6\,ms^{-1}$ and the BV in the radial artery was averaged at $0.44\,ms^{-1}$. This is for a systolic pressure (during heart contraction) and diastolic pressure (during heart relaxation) of $127 \sim 171\,mmHg$ and $54 \sim 72\,mmHg$ and a heart rate between $87 \sim 91\,BPM$.

After converting all units to SI (and taking the midpoint of all measured parameters):

```
L = 0.000005  #m
N = 128  #number of grid points
omega = 1.4833   #s^-1 = BPM/60
nu = 0.0032965  #Pa*s
rho = 1050  #kg*m^-3
G = -11465.69  #Pa = 133*mmHg = diastolic - systolic
T = 100  #final time
dt = 0.001  #time step size
```

For these parameters, the system becomes stiff and the adaptive time-stepping RK45 scheme fails to produce a solution, so instead SciPy's BDF scheme is used. Using these new conditions, we can solve for $\mathbf{u}$:

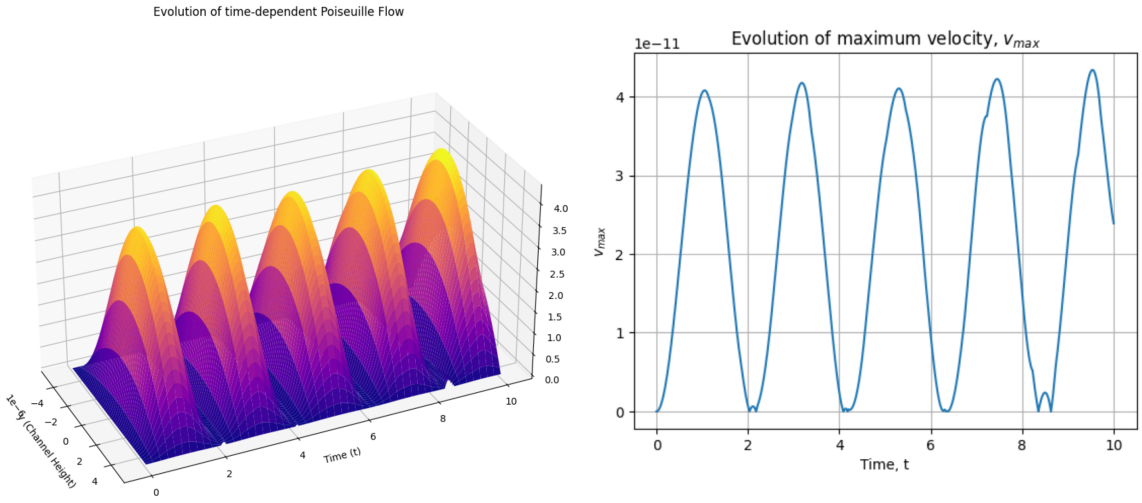

Figure 2: Evolution of velocity profiles of blood through capillaries

The capillary simulation gives maximum velocities of around $4.1\,cms^{-1}$, which is of the correct magnitude, but slightly below the given range. This could be for a number of reasons.

The parameters L, ρ and ν were taken as averages for typical radii, densities and dynamic viscosities seen in capillaries, which can introduce some error. However, the likely culprit is that the no-slip boundary conditions that were enforced are slowing the flow down by a much greater amount than is seen physically. This is especially relevant, as boundary effects are amplified when dealing with capillary flow, due to the small length scale in the $\mathbf{e}_y$ direction.

There is also what appears to be a reflection in the x axis, which is due to artifacts induced by the small domain size, as simulations run for similarly scaled values but on larger domains (Fig.1) do not show these errors.

The arterial simulation shows a much more stable plot of the velocity, which is due to the larger length scale in the $\mathbf{e}_y$ direction. We can see that the velocity, given in cms^{-1} is within the specified range of the experiment, which indicates an accurate simulation. Because the diameter is 3 orders of magnitude larger than for capillaries, boundary effects effect the system much less (although still play a major role in real life). Despite this, the simulation still managed to accurately show the oscillating velocity profile we expect from the pressure function and still exhibits the characteristic quadratic form when taken

at some characteristic time as in (Fig.1).

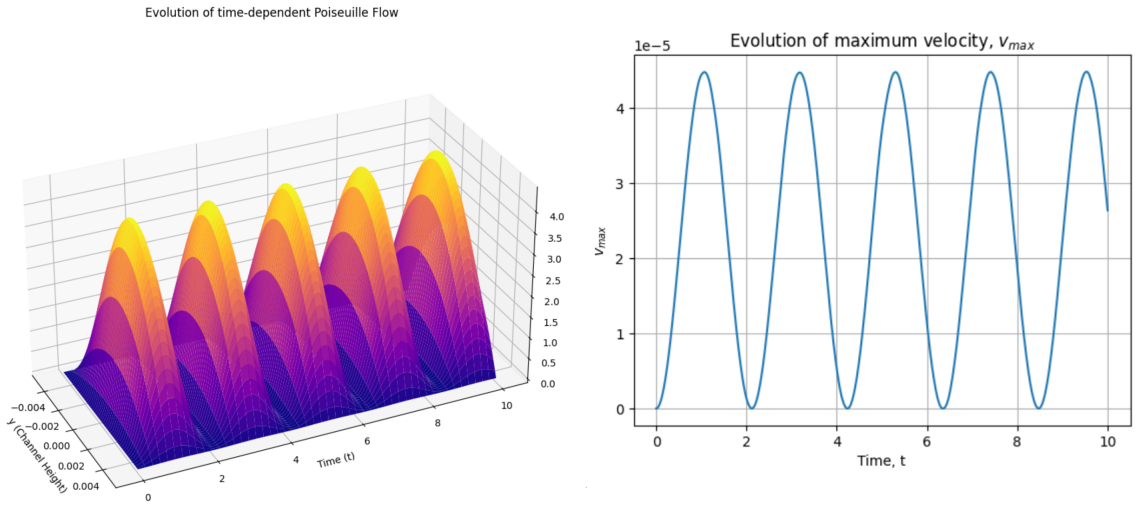

Figure 3: Evolution of velocity profiles of blood through the radial artery

IV. Conclusion

The simulation was very successful in simulating startup flow for a general pipe, as shown by the initial velocity increase in (Fig.1), and exhibited the expected smooth oscillations in velocity and therefore volume flux. Further refinements could be made however to add a partial slip condition to the boundaries to more accurately simulate capillary flow, and to increase the resolution of the system possibly using parallelisation, to allow the small-scale flow to be computed accurately and stably and in a reasonable time.

References

[1] Lee, S.-S., Choi, R.-G., Kim, W.-T., Shin, M.-W., Choi, J.-G., Hasan, M., Jung, B. (2024). Characteristics of blood flow velocity in the radial artery and finger capillaries using magnetoplethysmogram and photoplethysmogram. AIP Advances, 14(2). https://doi.org/10.1063/9.0000793 .

[2] Secomb, T. W., Pries, A. R. (2013). Blood viscosity in microvessels: experiment and theory. Comptes Rendus Physique, 14(6), 470–478. https://doi.org/10.1016/j.crhy.2013.04.002.

[3] Lee, S.-S., Choi, R.-G., Kim, W.-T., Shin, M.-W., Choi, J.-G., Hasan, M., Jung, B. (2024). Characteristics of blood flow velocity in the radial artery and finger capil-

laries using magnetoplethysmogram and photoplethysmogram. AIP Advances, 14(2). https://doi.org/10.1063/9.0000793